Lab 4: Temporal Difference Learning

Morgan Cooper
MSDS 684 — Reinforcement Learning

April 5, 2026

Section 1: Project Overview

This lab builds upon Monte Carlo (MC) and Dynamic Programming (DP) by introducing Temporal Difference (TD) Learning. TD operates on-policy, meaning it updates every step instead of at the end of each episode. It does so by using bootstrapping, which involves updating value estimates without a complete model, which leads to faster training/learning. This lab uses the same agent structure while only differing by one line of code: a SARSA agent uses on-policy, and a Q-learning agent updates off-policy. This alteration leads to two very different approaches in CliffWalking.

The environment used in the lab follows:

CliffWalking-v1 (Gymnasium):

- State space: 48 discrete states (4 rows $\times$ 12 columns), integer 0–47
- Action space: 4 discrete actions — up, right, down, left
- Rewards: -1 per step, -100 for cliff (resets to start)
- Terminal condition: reaching state 47 (bottom-right goal)

Since SARSA is updated based on the action it takes, I estimate that it will follow the least risky path in the CliffWalking environment. The Q-learning agent however will take much higher risk to find the most optimal solution. I anticipate training to be more negative for the Q-learning agent but to have the more optimal policy after training is complete.

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/temporal-difference-learning`

Implementation Summary

I implemented SARSA and Q-learning on CliffWalking-v1. For each of the agents I used an epsilon greedy policy with epsilon=0.1, alpha=0.5, gamma=1.0, and a Q-table from a numpy array. For each I ran 30 independent experiments based on 30 different seeds. Each experiment was run over 1000 episodes and plotted for visual inspection. Additionally, I experimented with different hyperparameters varying alpha (0.1, 0.3, 0.5) and epsilon-decay (constant, linear, exponential).

Key Results & Analysis

SARSA averaged -28.0 reward over last 100 episodes, whereas Q-learning averaged -51.6 as seen in Figure 1. SARSA was 23.5 points better during training because it avoids the cliff. On the other hand, Q-learning falls off the cliff more during training due to exploration along the cliff's edge.

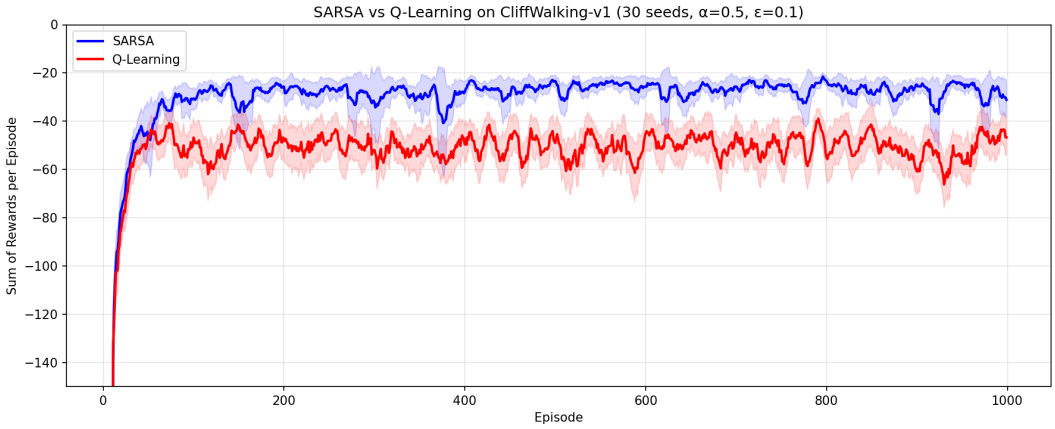

Figure 1: SARSA vs Q-learning learning curves averaged over 30 seeds with 95% confidence intervals. Sum of rewards per episode over 1000 training episodes with $\alpha = 0.5$ and $\epsilon = 0.1$.

When we look at the learned policies for SARSA and Q-learning we can see very different paths (Figure 2 & Figure 3). SARSA arrows go up from start, right across the top row, then down to goal. SARSA takes the safest path of 17 steps and scores a reward of -17 to stay as far as possible from the cliff. Q-learning arrows go right along row 2, directly above the cliff edge, then down to goal. This is the optimal path, taking only 13 steps and scoring a reward of -13.

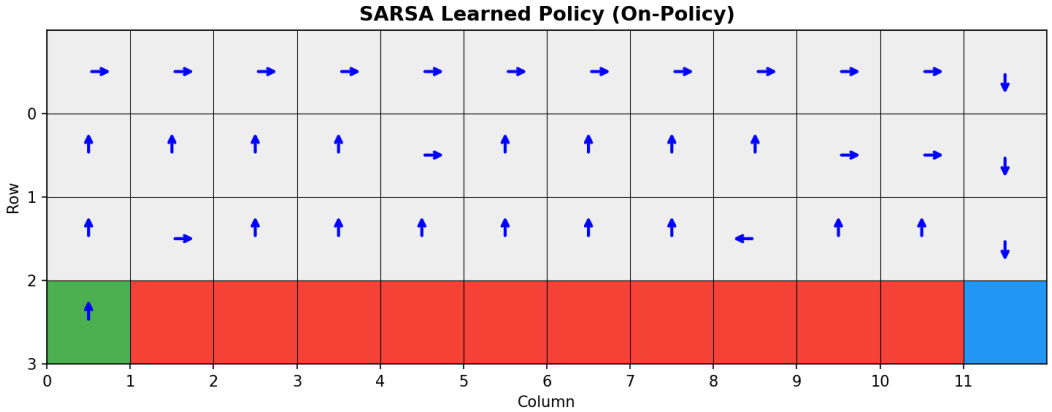

Figure 2: SARSA greedy policy arrows showing the learned action at each grid state after 2000 episodes of training. Green = start, blue = goal, red = cliff.

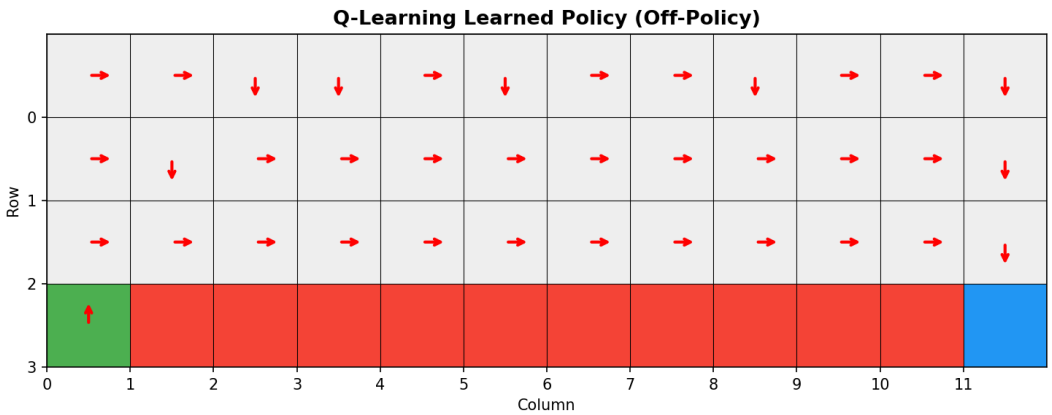

Figure 3: Q-learning greedy policy arrows showing the learned action at each grid state after 2000 episodes of training. Green = start, blue = goal, red = cliff.

When experimenting with differing epsilon decay schedules (Figure 4), exponential decay improved Q-learning the most, changing from -50.5 to -16.8. This happens because epsilon drops close to zero quickly and the agent stops falling off the cliff as much. Linear epsilon decay still improved the policy over time but much slower. Finally, exponential decay seemed to work the best given enough training time. With enough time, Q-learning eventually beats SARSA because the optimal cliff path becomes safe to follow after epsilon has decayed enough.

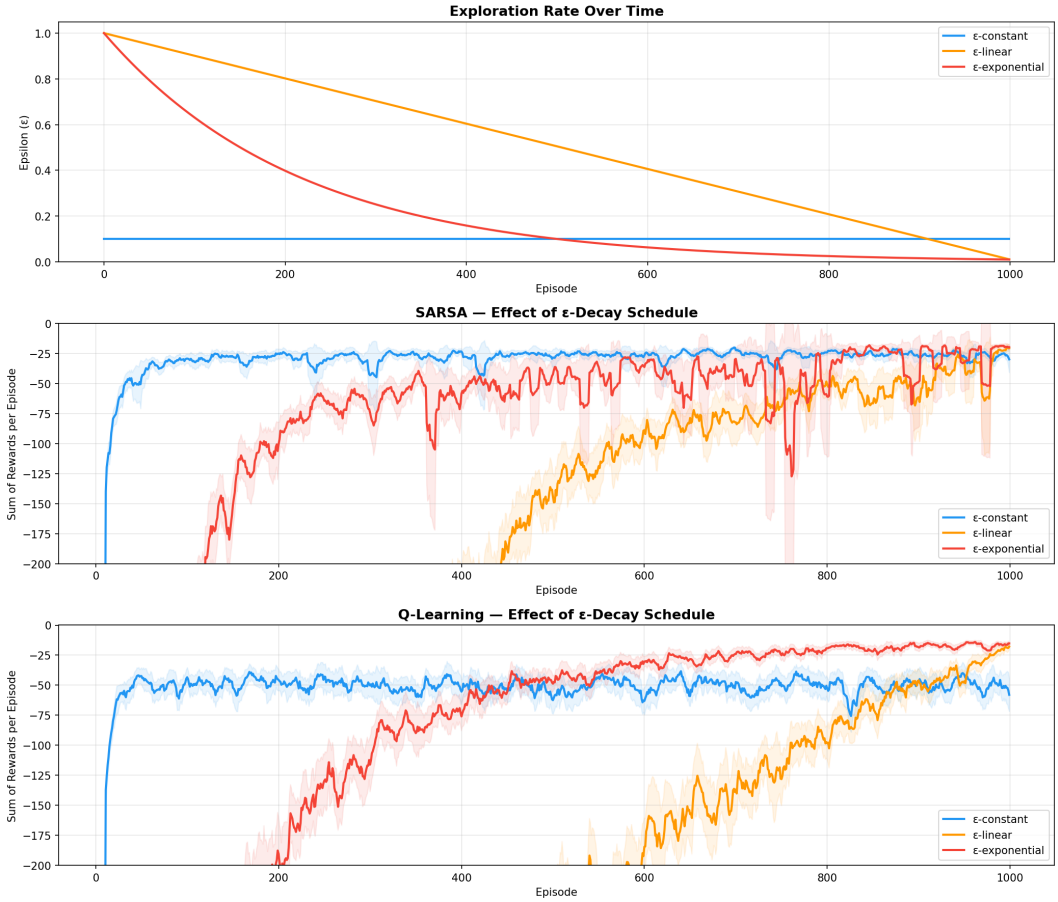

Figure 4: Epsilon decay schedule comparison. Top: epsilon value over time for each schedule. Middle: SARSA reward curves. Bottom: Q-learning reward curves. All averaged over 20 seeds.

Section 3: AI Use Reflection

Initial Interaction

TODO: Write initial interaction here.

Iteration Cycle

TODO: Write iteration cycles here.

Critical Evaluation

TODO: Write critical evaluation here.

Learning Reflection

TODO: Write learning reflection here.

Section 4: Speaker Notes

- **Problem:** TODO
- **Method:** TODO
- **Design choice:** TODO
- **Key result:** TODO
- **Insight:** TODO
- **Challenge:** TODO
- **Connection:** TODO

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
3. OpenAI. (2025). ChatGPT [Large language model]. `https://chat.openai.com`